

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA1402

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO5: *Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code.(BL3)*

Assessment Tool Weightage:20%

QUESTIONS: 15

Total Marks:50

Annexure A

Viva Voce

Code of Conduct:

*I _Monika.R(192324281)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Monika.R

Annexure A

Viva Voce

QUESTIONS

1. Compare top-down and bottom-up parsing for arithmetic expressions. Which is better and why?
2. A grammar contains left recursion. Explain how it impacts recursive descent parsing.
3. A parser generates multiple parse trees for one input. How will you resolve it?
4. Why is left factoring required in predictive parsing?
5. How does operator precedence affect parse tree generation?
6. If parentheses are missing in an expression, explain how the parser detects it.
7. Evaluate why LL(1) parsing is considered deterministic.
8. If ϵ belongs to FIRST(A), how does it influence FOLLOW(A)?
9. A grammar has overlapping FIRST sets. Analyze the effect on LL(1) parsing.
10. Why is FOLLOW needed only when ϵ appears? Explain with logic.
11. Why is grammar augmentation necessary before LR parsing?
12. How do closure and goto functions help build LR item sets?
13. What are the SLR parsing steps?
14. Why is three-address code preferred for optimization?
15. Compare machine-independent and machine-dependent optimization.

1. Compare top-down and bottom-up parsing for arithmetic expressions. Which is better and why?

Top-down parsing starts from the start symbol and derives the input. Bottom-up parsing starts from the input and reduces it to the start symbol. Bottom-up parsing is better for arithmetic expressions because it handles operator precedence, associativity, and left recursion effectively.

2. A grammar contains left recursion. Explain how it impacts recursive descent parsing.

Left recursion causes infinite recursion in recursive descent parsing because the parser repeatedly calls the same non-terminal without consuming input. Therefore, left recursion must be removed.

3. A parser generates multiple parse trees for one input. How will you resolve it?

Multiple parse trees indicate an ambiguous grammar. It can be resolved by rewriting the grammar and specifying operator precedence and associativity.

4. Why is left factoring required in predictive parsing?

Left factoring removes common prefixes from productions. It allows the parser to select the correct production using **one lookahead symbol**.

5. How does operator precedence affect parse tree generation?

Operator precedence determines which operation is performed first. For example, in $a + b * c$, multiplication has higher precedence, so the tree represents $a + (b * c)$.

6. If parentheses are missing in an expression, explain how the parser detects it.

The parser expects a matching closing parenthesis according to the grammar. If it reaches the end of input or encounters an unexpected symbol instead of $)$, it reports a **syntax error**.

7. Evaluate why LL(1) parsing is considered deterministic.

LL(1) parsing uses one lookahead symbol to select exactly one production. Therefore, there is no backtracking when the grammar satisfies LL(1) conditions.

8. If ϵ belongs to FIRST(A), how does it influence FOLLOW(A)?

If $\epsilon \in \text{FIRST}(A)$, A can produce an empty string. Therefore, when A occurs in a production, symbols that follow A can become part of FOLLOW(A). FOLLOW helps determine when to use the ϵ -production.

9. A grammar has overlapping FIRST sets. Analyze the effect on LL(1) parsing.

Overlapping FIRST sets cause conflicts because the parser cannot decide which production to choose using one lookahead symbol. Hence, the grammar is not LL(1).

10. Why is FOLLOW needed only when ϵ appears? Explain with logic.

FOLLOW is needed when a non-terminal can derive ϵ . It tells the parser when the non-terminal can be skipped and the next input symbol can be matched.

11. Why is grammar augmentation necessary before LR parsing?

Grammar augmentation adds a new start symbol, such as $S' \rightarrow S$. It provides a clear acceptance condition for the LR parser.

12. How do closure and goto functions help build LR item sets?

Closure adds required productions to an item set. **Goto** moves the dot over a grammar symbol to create a new item set. Together they construct the LR item collection.

13. What are the SLR parsing steps?

1. Augment the grammar.
2. Construct LR(0) items.
3. Apply closure and goto.
4. Construct ACTION and GOTO tables.
5. Use FOLLOW sets for reduce actions.
6. Parse the input using the stack.
7. Accept when the start production is completed.

14. Why is three-address code preferred for optimization?

Three-address code uses simple instructions and temporary variables. It makes **code analysis, optimization, and code generation easier**.

Example:

$t1 = b * c$

$t2 = a + t1$

15. Compare machine-independent and machine-dependent optimization.

Machine-independent optimization does not depend on a specific processor, e.g., constant folding and dead-code elimination.

Machine-dependent optimization depends on the target hardware, e.g., register allocation and instruction scheduling.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	25	Demonstrates deep understanding of concepts with complete clarity	Explains concepts correctly with minor gaps	Partial understanding with errors or omissions	Unable to explain basic concepts
Application	25	Effectively applies concepts to new situations; solves problems logically	Applies concepts with minor errors	Limited ability to apply concepts; requires guidance	Unable to apply concepts effectively
Analytical Thinking	20	Critically analyzes scenarios with strong justification and reasoning	Provides reasonable analysis with some justification	Superficial analysis with weak reasoning	No analytical reasoning demonstrated
Communication	15	Communicates ideas clearly, confidently, and with appropriate terminology	Communicates clearly but with minor hesitation	Communication lacks clarity or structure	Unable to communicate ideas effectively
Response to Questions	15	Responds accurately and confidently, extending answers with depth	Responds correctly but with limited depth	Responds partially; requires prompting	Unable to respond to questions effectively